

## Architecture of .Net platform

**.NET is a framework** to develop software applications. It is designed and developed by Microsoft and the first beta version released in 2000.

This framework contains a large number of class libraries known as Framework Class Library (FCL). The software programs written in .NET are executed in the execution environment, which is called CLR (Common Language Runtime). These are the core and essential parts of the .NET framework.

This framework provides various services like memory management, networking, security, memory management, and type-safety.

The .Net framework consists of developer tools, programming languages, and libraries to build desktop and web applications. It is also used to build websites, web services, and games.

The .Net framework was meant to create applications, which would run on the Windows Platform. **The first version of the .Net framework was released in the year 2002. The version was called .Net framework 1.0.** The Microsoft .Net framework has come a long way since then, and the **Current version is .Net Framework 4.7.2.**

The Microsoft .Net framework can be used to create both – **Form-based** and **Web-based** applications. Web services can also be developed using the .Net framework.

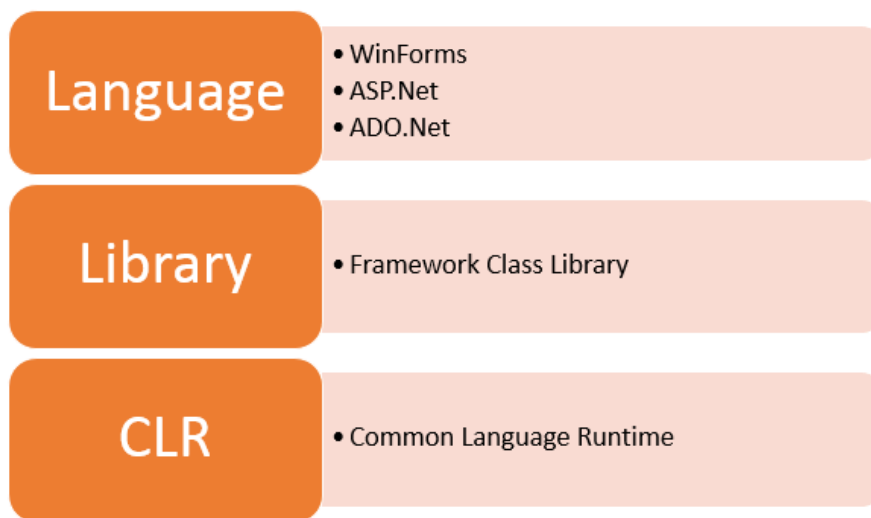
**The .Net Framework supports more than 60 programming languages** such as C#, F#, VB.NET, J#, VC++, JScript.NET, APL, COBOL, Perl, Oberon, ML, Pascal, Eiffel, Smalltalk, Python, Cobra, ADA, etc.

**. NET Framework is part of the . NET platform, a collection of technologies for building apps for Linux, macOS, Windows, iOS, Android, and more.**

# .Net Framework Architecture

**.Net Framework Architecture** is a programming model for the .Net platform that provides an execution environment and integration with various programming languages for simple development and deployment of various Windows and desktop applications. It consists of class libraries and reusable components.

The basic architecture of the .Net framework is as shown below.



## **.Net Framework Architecture Diagram**

## Components of .NET Framework

There are following components of .NET Framework:

1. CLR (Common Language Runtime)
2. CTS (Common Type System)
3. BCL (Base Class Library)
4. CLS (Common Language Specification)
5. FCL (Framework Class Library)
6. .NET Assemblies
7. XML Web Services

8. Window Services
9. Core Languages (WinForms, ASP.NET, and ADO.NET), and
10. Other Modules (WCF, WPF, WF, Card Space, LINQ, Entity Framework, Parallel LINQ, Task Parallel Library, etc.)

The architecture of .Net framework is based on the following key components;

## 1. Common Language Runtime

The “Common Language Infrastructure” or CLI is a platform in .Net architecture on which the .Net programs are executed.

The CLI has the following key features:

**Exception Handling** – Exceptions are errors which occur when the application is executed.

Examples of exceptions are:

- If an application tries to open a file on the local machine, but the file is not present.
- If the application tries to fetch some records from a database, but the connection to the database is not valid.

**Garbage Collection** – Garbage collection is the process of removing unwanted resources when they are no longer required.

Examples of garbage collection are

- A File handle which is no longer required. If the application has finished all operations on a file, then the file handle may no longer be required.
- The database connection is no longer required. If the application has finished all operations on a database, then the database connection may no longer be required.

## Working with Various programming languages –

As noted in an earlier section, a developer can develop an application in a variety of .Net programming languages.

1. **Language** – The first level is the programming language itself, the most common ones are VB.Net and C#.

Level 1

VB.Net

C#

2. **Compiler** – There is a compiler which will be separate for each programming language. So underlying the VB.Net

Level 2

VB.Net  
Compiler

C#  
Compiler

3. **Common Language Interpreter** – This is the final layer in .Net which would be used to run a .net program developed in any programming language. So the subsequent compiler will send the program to the CLI layer to run the .Net application.

Level 3

Common  
Language  
Interpreter

## 2. Class Library

The .NET Framework includes a set of standard class libraries. A class library is a collection of methods and functions that can be used for the core purpose.

For example, there is a class library with methods to handle all file-level operations. So there is a method which can be used to read the text from a file. Similarly, there is a method to write text to a file.

Most of the methods are split into either the System.\* or Microsoft.\* namespaces. (The asterisk \* just means a reference to all of the methods that fall under the System or Microsoft namespace)

A namespace is a logical separation of methods. We will learn these namespaces more in detail in the subsequent chapters.

### 3. Languages

The types of applications that can be built in the .Net framework is classified broadly into the following categories.

**WinForms** – This is used for developing Forms-based applications, which would run on an end user machine. Notepad is an example of a client-based application.

**ASP.Net** – This is used for developing web-based applications, which are made to run on any browser such as Internet Explorer, Chrome or Firefox.

- The Web application would be processed on a server, which would have Internet Information Services Installed.
- Internet Information Services or IIS is a Microsoft component which is used to execute an Asp.Net application.
- The result of the execution is then sent to the client machines, and the output is shown in the browser.

**ADO.Net** – This technology is used to develop applications to interact with Databases such as Oracle or Microsoft SQL Server.

Microsoft always ensures that .Net frameworks are in compliance with all the supported Windows operating systems.

# .Net Framework Design Principle

Now in this .Net Architecture tutorial, we will learn the design principles of .Net framework. The following design principles of the .Net framework is what makes it very relevant to create .Net based applications.

**1) Interoperability** – The .Net framework provides a lot of backward support. Suppose if you had an application built on an older version of the .Net framework, say 2.0. And if you tried to run the same application on a machine which had the higher version of the .Net framework, say 3.5. The application would still work. This is because with every release, Microsoft ensures that older framework versions gel well with the latest version.

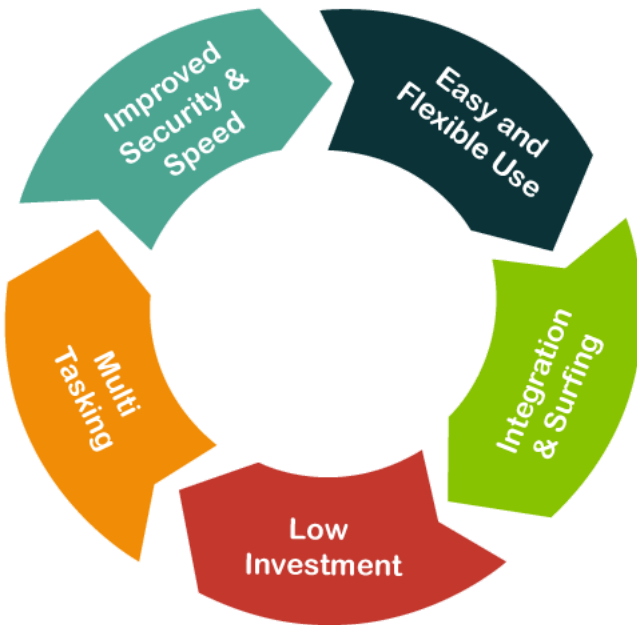
**2) Portability** – Applications built on the .Net framework can be made to work on any Windows platform. And now in recent times, Microsoft is also envisioning to make Microsoft products work on other platforms, such as iOS and Linux.

**3) Security** – The .NET Framework has a good security mechanism. The inbuilt security mechanism helps in both validation and verification of applications. Every application can explicitly define their security mechanism. Each security mechanism is used to grant the user access to the code or to the running program.

**4) Memory management** – The Common Language runtime does all the work or memory management. The .Net framework has all the capability to see those resources, which are not used by a running program. It would then release those resources accordingly. This is done via a program called the “Garbage Collector” which runs as part of the .Net framework. The garbage collector runs at regular intervals and keeps on checking which system resources are not utilized, and frees them accordingly.

**5) Simplified deployment** – The .Net framework also have tools, which can be used to package applications built on the .Net framework. These packages can then be distributed to client machines. The packages would then automatically install the application.

# Characteristics of .NET Framework



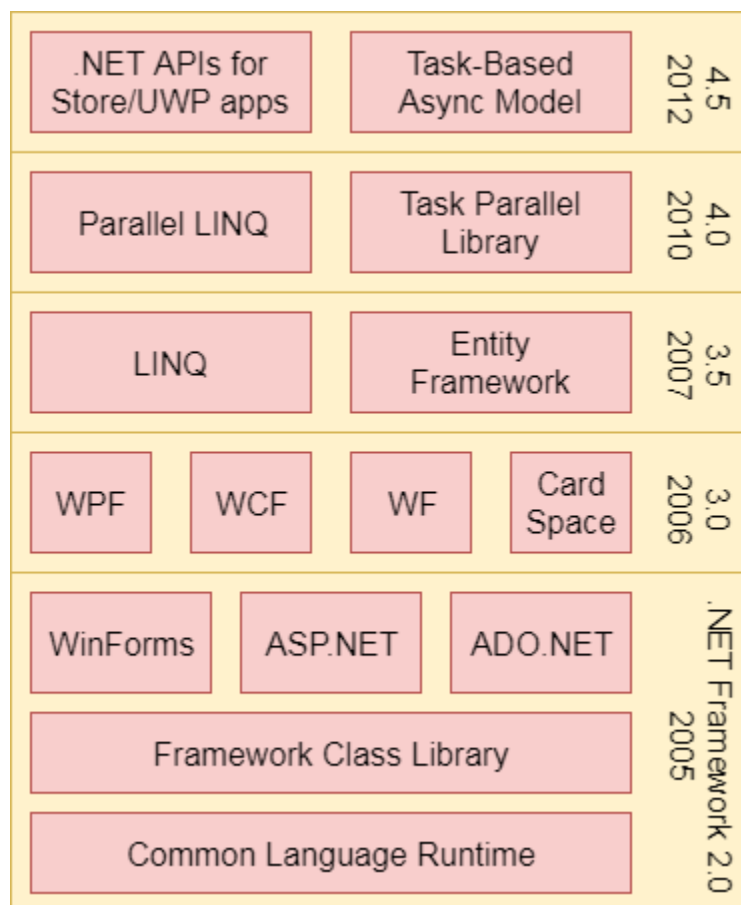
1. CLR (Common Language Runtime)
2. Namespace - Predefined class and function
3. Metadata and Assemblies
4. Application domains
5. It helps to configure and deploy the .net application
6. It provides form and web-based services
7. NET and ASP.NET AJAX
8. LINQ
9. Security and Portability
10. Interoperability
11. It provides multiple environments for developing an application

## Entity Framework

It is an **ORM** based open source framework which is used to work with a database using .NET objects. It eliminates a lot of developers effort to handle the database. It is Microsoft's recommended technology to deal with the database.

## . NET Framework Design Goals

- Component Infrastructure. Prior to the existence of COM technology, Microsoft developers had no simple way to integrate binary libraries without referring to or altering their source code. ...
- Language Integration. ...
- Internet Interoperation. ...
- Simple Development. ...
- Simple Deployment. ...
- Reliability. ...
- Security.





# CLR-(Common Language Runtime )

.NET CLR is a runtime environment that manages and executes the code written in any .NET programming language. CLR is the virtual machine component of the .NET framework. That language's compiler compiles the source code of applications developed using .NET compliant languages into CLR's intermediate language called MSIL, i.e., Microsoft intermediate language code. This code is platform-independent.

Metadata is also generated during compilation and MSIL code and stored in a file known as the Manifest file. This metadata is generally about members and types required by CLR to execute MSIL code.

A just-in-time compiler component of CLR converts MSIL code into native code of the machine. This code is platform-dependent. CLR manages memory, threads, exceptions, code execution, code safety, verification, and compilation.

## The main components of CLR are:

- Common type system
- Common language specification
- Garbage Collector
- Just in Time Compiler
- Metadata and Assemblies

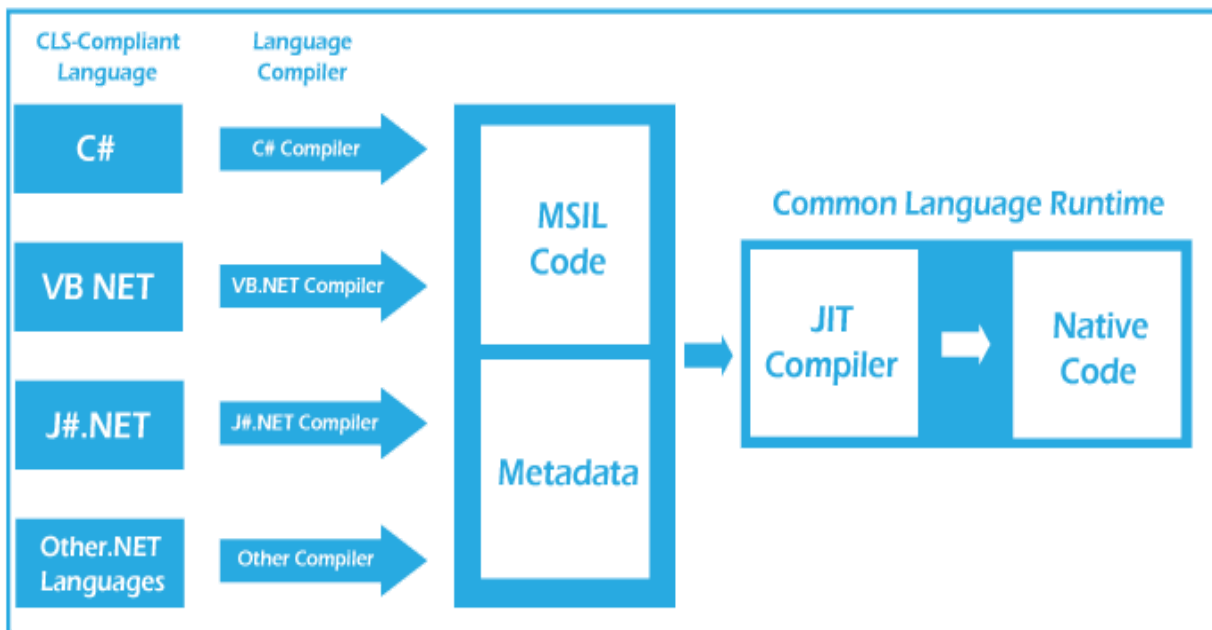
## .NET CLR Versions

### .NET version CLR version

|     |     |
|-----|-----|
| 1.0 | 1.0 |
| 1.1 | 1.1 |
| 2.0 | 2.0 |
| 3.0 | 2.0 |

|     |     |
|-----|-----|
| 3.5 | 2.0 |
| 4.0 | 4   |
| 4.5 | 4   |
| 4.6 | 4   |
| 4.6 | 4   |

The following figure shows the conversion of source code into native code.



Execution of a .NET Application

Converting Source Code into Native Code



The above figure converts code into native code, which the CPU can execute.

## .NET CLR Functions

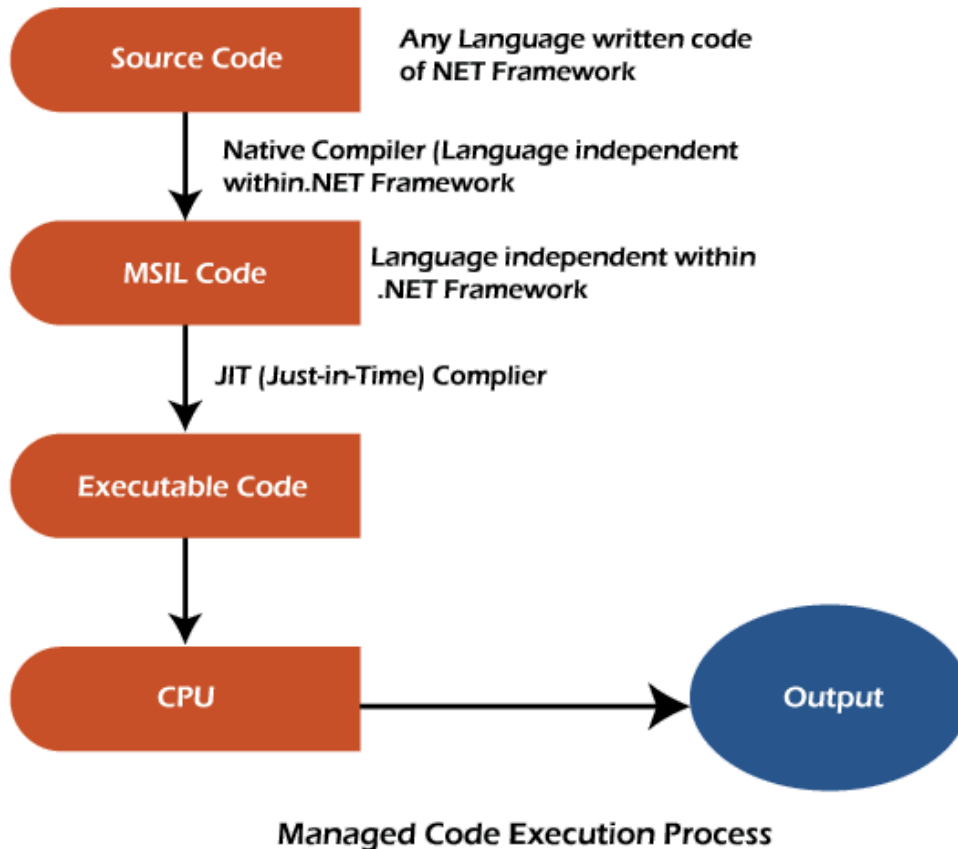
Following are the functions of the CLR.

- It converts the program into native code.
- Handles Exceptions
- Provides type-safety
- Memory management
- Provides security
- Improved performance
- Language independent
- Platform independent
- Garbage collection
- Provides language features such as inheritance, interfaces, and overloading for object-oriented programs.

The code that runs with CLR is called managed code, whereas the code outside the CLR is called unmanaged code. The CLR also provides an Interoperability layer, which allows both the managed and unmanaged codes to interoperate.

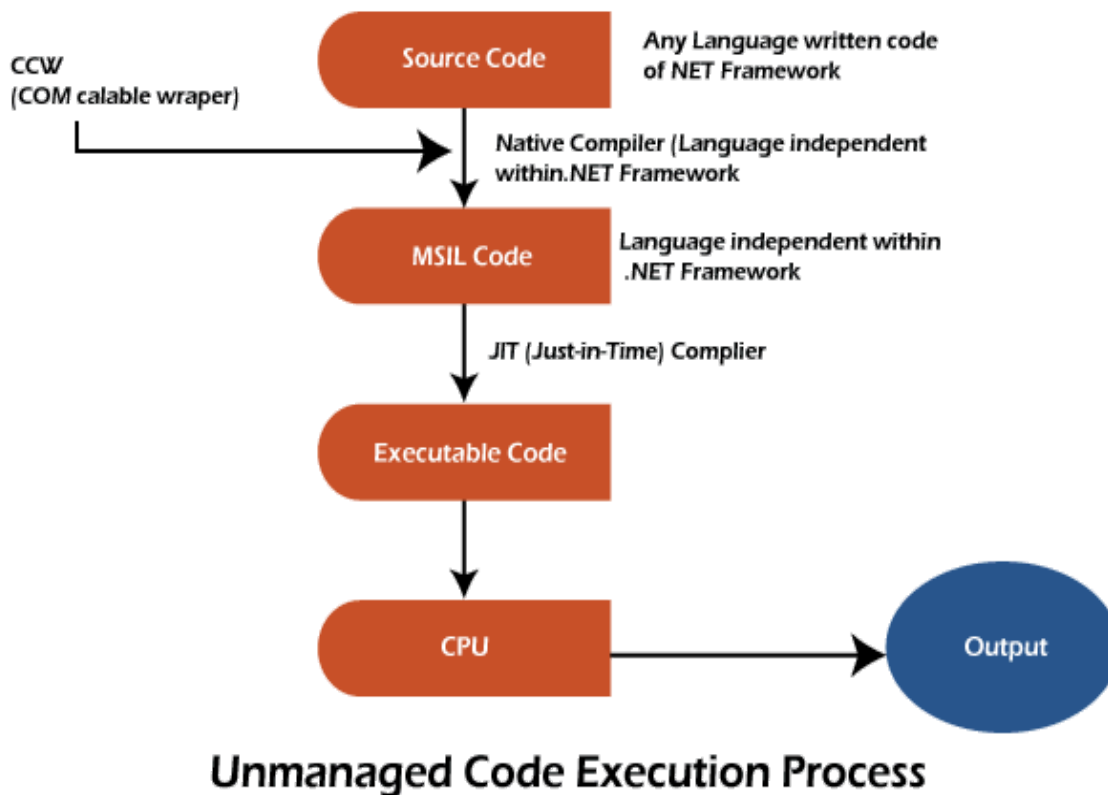
## **1. Managed code:**

Any language that is written in the .NET framework is managed code. Managed code use CLR, which looks after your applications by managing memory, handling security, allowing cross-language debugging, etc. The process of managed code is shown in the figure:



## 2. Unmanaged code:

The code developed outside the .NET framework is known as unmanaged code. Applications that do not run under the control of the CLR are said to be unmanaged. Certain languages such as C++ can be used to write such applications, such as low-level access functions of the operating system. Background compatibility with VB, ASP, and COM are examples of unmanaged code. This code is executed with the help of wrapper classes. The unmanaged code process is shown below:



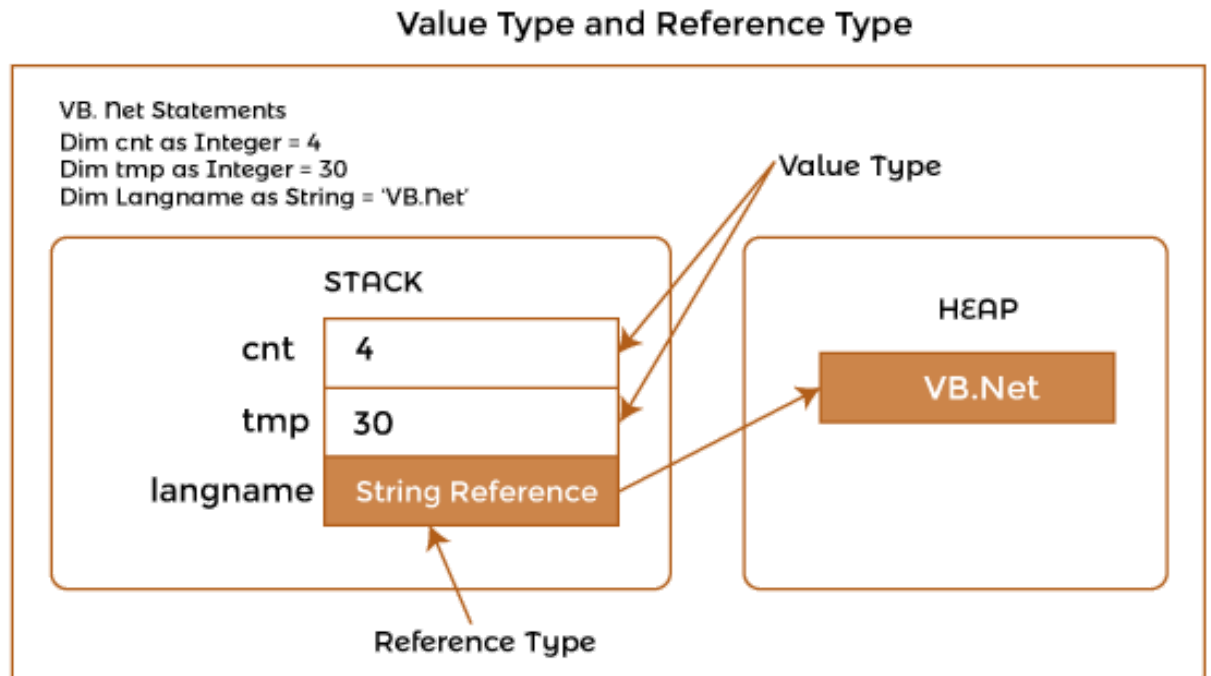
## 1. Common type system:

CTS provides guidelines for declaring, using, and managing data types at runtime. It offers cross-language communication. For example, VB.NET has an integer data type, and C# has an int data type for managing integers. After compilation, Int32 is used by both data types. So, CTS provides the data types using managed code. A common type system helps in writing language-independent code.

**It provides two categories of Types.**

1. **Value Type:** A value type stores the data in memory **allocated on the stack** or inline in a structure. This category of Type holds

the data directory. If one variable's value is copied to another, both the variables store data independently. It can be of inbuilt-in types, user-defined, or enumerations types. Built-in types are primitive data types like numeric, Boolean, char, and date. Users in the source code create user-defined types. An enumeration refers to a set of enumerated values represented by labels but stored as a numeric type.



- 2. Reference Type:** A Reference type stores a reference to the value of a memory address and is **allocated on the heap**. Heap memory is used for dynamic memory allocation. Reference Type does not hold actual data directly but holds the **address of data**. Whenever a reference type object is made, it copies the address and not actual data. Therefore two variables will refer to the same data. If data of one Reference Type object is changed, the same is reflected for the other object. Reference types can be self-describing types, pointer types, or interference types. The self-describing types may be string, array, and class types that store metadata about themselves.

## **2. Common Language Specification (CLS):**

Common Language Specification (CLS) contains a set of rules to be followed by all NET-supported languages. The common rules make it easy to implement language integration and help in cross-language inheritance and debugging. Each language supported by NET Framework has its own syntax rules. But CLS ensures interoperability among applications developed using NET languages.

### **3. Garbage Collection:**

**Garbage Collector is a component of CLR that works as an automatic memory manager.** It helps manage memory by automatically allocating memory according to the requirement. It allocates heap memory to objects. When objects are not in use, it reclaims the memory allocated to them for future use. It also ensures the safety of objects by not allowing one object to use the content of another object.

Garbage collection is the process of removing unwanted resources when they are no longer required.

Examples of garbage collection are

- A File handle which is no longer required. If the application has finished all operations on a file, then the file handle may no longer be required.
- The database connection is no longer required. If the application has finished all operations on a database, then the database connection may no longer be required.

### **4. Just in Time (JIT) Compiler:**

JIT Compiler is an important component of CLR. It converts the MSIL code into native code (i.e., machine-specific code). The .NET program is compiled either explicitly or implicitly

In implicit compilation, the **program is compiled twice**. The **source code is compiled into Microsoft Intermediate Language (MSIL)** during the first compilation process. The **MSIL code is converted into native code** in the second compilation process. This process is called JIT compilation.

There are three types of JIT compilers -Pre, Econo, and **Normal**. Pre JIT Compiler compiles **entire** MSIL code into native code before execution. Econo JIT Compiler compiles **only those parts of MSIL code required during execution** and removes those parts that are not required anymore. Normal JIT Compiler also compiles only those parts of MSIL code required during

## **5. Metadata:**

A Metadata is a binary information about the program, either stored in a CLR Portable Executable file (PE) along with MSIL code or in the memory. During the execution of MSIL, metadata is also loaded into memory for proper interpretation of classes and related. Information used in code. So, metadata helps implement code in a language-neutral manner or achieve language interoperability.

## **6. Assemblies:**

**Assembly** is an important concept in C#. It is a collection of code files that are compiled into an executable or a **Dynamic Link Library (DLL)**. Depending on their location and intended use, assemblies can be divided into many categories. We will examine the various assembly types in C# in this article.

**3 types of Assemblies - Private, Shared, Satellite**



## Private Assemblies:

An Assembly that is solely used by one application is referred to as a Private Assembly. It is typically found in the directory for the application or a subdirectory of the directory for the programme. Private Assemblies are not intended to be shared with other applications. They are used to store application-specific code and resources.

Private Assemblies are created when an application is compiled. When an application is compiled, all the code files and resources that are used by the application are compiled into a single assembly. The application's directory or a subdirectory of the application's directory will thereafter contain this assembly.

Private Assemblies are simple to deploy and use. They don't need any extra installation or setting. They are automatically loaded by the .NET runtime when the application starts.

## Shared Assemblies:

An assembly that is used by several programmes is referred to as a shared assembly. It is typically found in the **Global Assembly Cache (GAC)** or a common directory. Multiple applications are supposed to share a shared assembly. They are used to store resources and code that are shared by various applications.

Shared Assemblies are created using the strong name tool (***sn.exe***). A digital signature for the assembly is applied using the strong name tool. The digital signature guarantees that the assembly is genuine and unaltered.

The **Global Assembly Cache (GAC)** houses shared assemblies. Shared assemblies are kept in the **GAC**, which serves as a central repository. The **GAC** location is in the Windows directory (***C:\Windows\assembly***). Shared assemblies are registered with the **GAC** using the **gacutil.exe** tool.

Shared assemblies require special configuration and installation. They cannot be simply copied to the application's directory. They need to be registered with the **GAC** using the ***gacutil.exe*** tool.

## Satellite Assemblies:

An assembly used to store regional resources is referred to as a Satellite Assembly. It is typically found in a subdirectory of the directory for the application or a subdirectory of the directory for the Shared Assembly. Localized resources like strings, pictures, and audio files are kept in satellite assemblies.

Satellite Assemblies are created using the **resgen.exe** tool. The **resgen.exe** tool is used to create a resource file (**.resx**) from a text file. The resource file is then compiled into a satellite assembly using the **al.exe** tool.

Satellite Assemblies are named using a specific naming convention. The naming convention for satellite assemblies is as follows:

## 1. <AssemblyName>.resources.dll

The <AssemblyName> part of the name is the name of the main assembly that the satellite assembly is associated with. The .NET runtime automatically loads satellite assemblies when the application first launches. The .NET runtime automatically selects the appropriate satellite assembly based on the user's culture settings.

## Dynamic Link Libraries (DLLs):

A **Dynamic Link Library (DLL)** is a type of assembly that contains code that can be used by multiple applications. Similar to shared assemblies, **DLLs** are created with the intention of being used by numerous applications. **DLLs**, however, differ from shared assemblies in that they do not have a distinctive name.

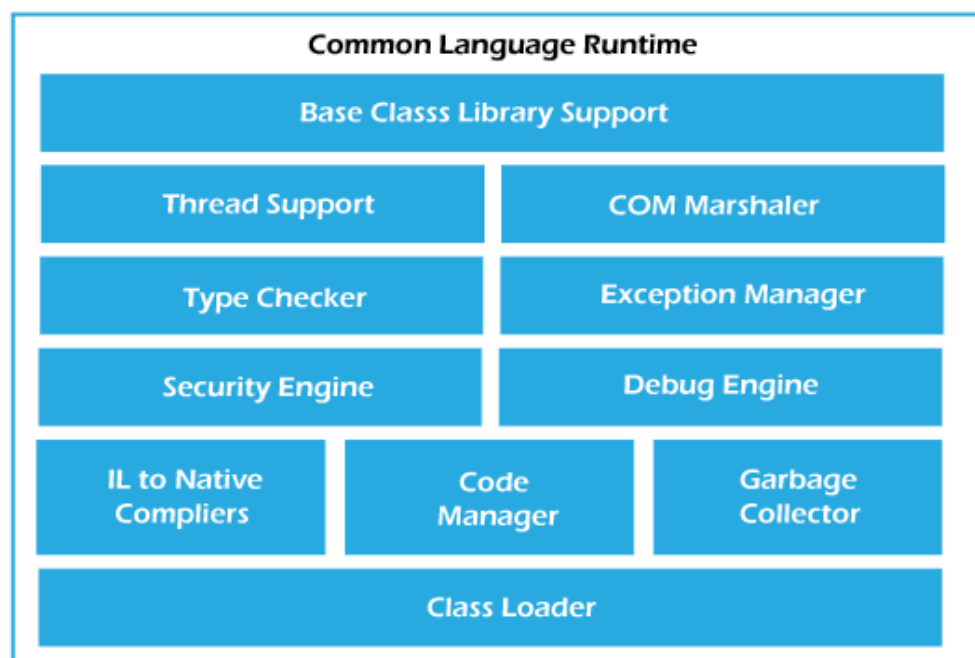
**DLLs** are kept in the directory of the application or a subdirectory of the directory of the application. They can be used by multiple applications by simply copying the **DLL** to the application's directory.

**DLLs** are created using the same tools that are used to create shared assemblies. However, **DLLs** do not require a strong name. They can be signed with a digital signature, but this is optional.

## CLR Structure

Following is the component structure of Common Language Runtime.

### Base Class Library Support



Components of the Common Language runtime/Architecture of CLR

It is a class library that supports classes for the .NET application.

### **Thread Support**

It manages the parallel execution of the multi-threaded application.

### **COM Marshaler**

It provides communication between the COM objects and the application.

### **Security Engine**

It enforces security restrictions.

### **Debug Engine**

It allows you to debug different kinds of applications.

### **Type Checker**

It checks the types used in the application and verifies that they match the standards provided by the CLR.

### **Code Manager**

It manages code at execution runtime.

### **Garbage Collector**

It releases the unused memory and allocates it to a new application.

### **Exception Handler**

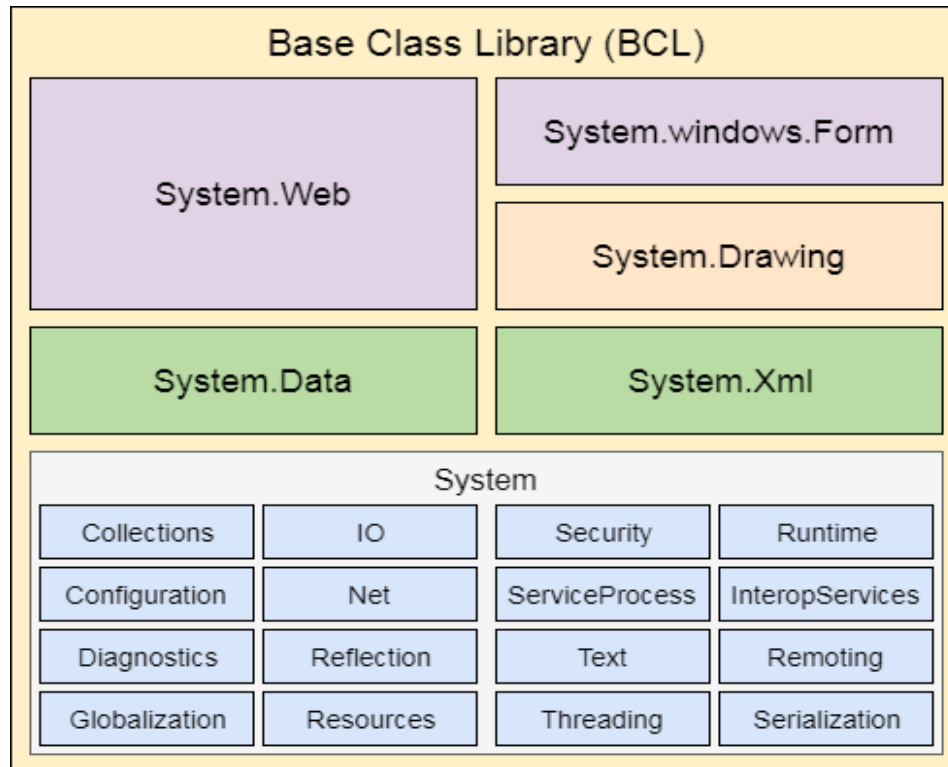
It handles the exception at runtime to avoid application failure.

### **ClassLoader**

It is used to load all classes at run time.

## FCL (Framework Class Library)

It is a standard library that is a collection of thousands of classes and used to build an application. The BCL (Base Class Library) is the core of the FCL and provides basic functionalities.



## WinForms

Windows Forms is a smart client technology for the .NET Framework, a set of managed libraries that simplify common application tasks such as reading and writing to the file system.

## ASP.NET

ASP.NET is a web framework designed and developed by Microsoft. It is used to develop websites, web applications, and web services. It provides a fantastic integration of HTML, CSS, and JavaScript. It was first released in January 2002.

## ADO.NET

ADO.NET is a module of .Net Framework, which is used to establish a connection between application and data sources. Data sources can be such as SQL Server and XML. ADO .NET consists of classes that can be used to connect, retrieve, insert, and delete data.

## WPF (Windows Presentation Foundation)

Windows Presentation Foundation (WPF) is a graphical subsystem by Microsoft for rendering user interfaces in Windows-based applications. WPF, previously known as "Avalon", was initially released as part of .NET Framework 3.0 in 2006. WPF uses DirectX.

## WCF (Windows Communication Foundation)

It is a framework for building service-oriented applications. Using WCF, you can send data as asynchronous messages from one service endpoint to another.

## LINQ (Language Integrated Query)

It is a query language, introduced in .NET 3.5 framework. It is used to make the query for data sources with C# or Visual Basics programming languages.

## What is .NET 3-Tier Architecture?

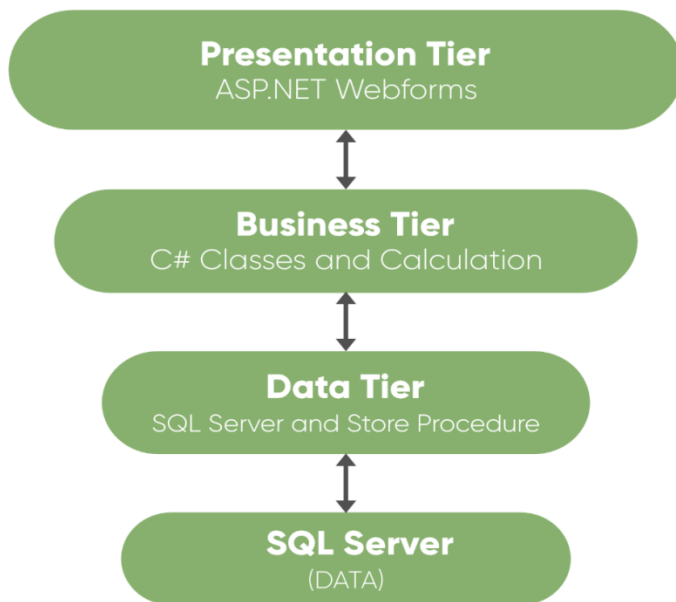
Three-layer architecture is dividing the project into three layers that are *User interface layer*, *business layer* and *data(database) layer* where we separate UI, logic, and data in three divisions.

Suppose the user wants to change the UI from windows to the phone than he has to only make change in UI layer, other layers are not affected by this change. Similarly, if the user wants to change the database then he has to only make a change in the data layer, rest everything remains the same.

## Why to use 3-Tier Architecture in Projects?

We use 3 Tier architecture to manage large projects. We can take an example: Suppose you have created 2000 entity code and you have coded all your code in only one layer. Now suppose you want to change from windows application to mobile or web application then again you have to write code for all 2000 entities or you want to change database provider than again you have to change database connection in all 2000 entities. Hence writing such a large amount of code is a time and money wasting. Also writing a large amount of code in one layer only makes new people difficult to understand that code.

So, to provide maintenance, flexibility, updation flexibility without affecting other parts, clean and understandable code we should use 3-Tier Architecture in Projects. This architecture basically contains **three layers**:



### 1. Presentation Layer

This is the top layer of architecture. The topmost level of application is the user interface. It is related to the user interface that is what the user sees. The main function of this layer is to translate tasks and results in something which the user can understand. It contains pages like web forms, windows form where data is presented to the user and use to take input from the user. The presentation layer

is the most important layer because it is the one that the user sees and good UI attracts the user and this layer should be designed properly.

## **2. Business Layer**

This is the middle layer of architecture. This layer involves C# classes and logical calculations and operations are performed under this layer. It processes the command, makes logical decisions and perform calculations. It also acts as a middleware between two surrounded layers that is presentation and data layer. It processes data between these two layers. This layer implements business logic and calculations. This layer also validates the input conditions before calling a method from the data layer. This ensures the data input is correct before proceeding, and can often ensure that the outputs are correct as well. This validation of input is called business rules.

## **3. Data Layer**

This layer is used to connect the business layer to the database or data source. It contains methods which are used to perform operations on database like insert, delete, update, etc. This layer contains stored procedures which are used to query database. Hence this layer establishes a connection with the database and performs functions on the database.

### **Advantages:**

- Updation to new graphical environments is easier and faster.
- Easy to maintain large and complex projects.
- It provides logical separation between presentation layer, business layer and data layer.
- By introducing application layer between data and presentation layer, it provides more security to database layer hence data will be more secure.
- You can hide unnecessary methods from the business layer in the presentation layer.
- Data provider queries can be easily updated and OOP's concept can be easily applied on project.

### **Disadvantages:**

- It is more complex than simple client-server architecture.

- It is difficult to build and time consuming.
- User should have good knowledge of object-oriented concepts.

## ● Explain the features of .NET

- Microsoft .NET provides a framework that facilitates designing and developing applications that are portable, scalable, and robust, and that can be executed in a distributed environment. It presents a platform and device-independent computing model in a managed environment.
- The Microsoft .NET framework provides a lot of features. Microsoft has designed the features of the .NET framework by using the technologies that are required by software developers to develop applications for modern as well as future business needs. The key features of .NET are:

- **1. Common Executive Environment:-**

- All .NET applications run under a common execution environment, called the Common Language Runtime. The CLR facilitates the interoperability between different .NET languages such as C#, Visual Basic, Visual C++, etc. by providing a common environment for the execution of code written in any of these languages.

- **2. Common Type System:-**

- The .NET framework follows types of systems to maintain data integrity across the code written in different .NET compliant programming languages. CTS ensures that objects of the programs that are written in different programming languages can communicate with each other to share data.
- CTS prevents data loss when a type in one language transfers data to its equivalent type in one language transfer data to its equivalent type in other languages. For example, CTS ensures that data is not lost while



transferring an integer variable of visual basic code to an integer variable of C# code.

- The common type system CTS defines a set of types and rules that are common to all languages targeted at the CLR. It supports both value and reference types. Value types are created in the stack and include all primitive types, structs, and enums. In contrast, reference types are created in the managed heap and include objects, arrays, collections, etc.

### ● **3. Multi-language support:-**

- .NET provides multi-language support by managing the compilers that are used to convert the source to intermediate language (IL) and from IL to native code, and it enforces program safety and security.
- The basis for multiple language support is the common type system and metadata. The basic data types used by the CLR are common to all languages. There are therefore no conversion issues with the basic integer, floating-point and string types.
- All languages deal with all data types in the same way. There is also a mechanism for defining and managing new types.

### ● **4. tool Support:-**

- The CLR works hand-in-hand with tools like visual studio, compilers, debuggers, and profilers to make the developer's job much simpler.

### ● **5. Security:-**

- The CLR manages system security through user and code identity coupled with permission checks. The identity of the code can be known and permission for the use of resources granted accordingly. This type of security is a major feature of .NET. The .NET framework also provides support for role-based security using windows NT accounts and groups.

- **6. Automatic Resource Management:-**

- The .NET CLR provides efficient and automatic resource management such as memory, screen space, network connections, database, etc. CLR invokes various built-in functions of .NET framework to allocate and de-allocate the memory of .NET objects.
- Therefore, programmers need not write the code to explicitly allocate and de-allocate memory to the program.

- **7. Easy and rich debugging support:-**

- The .NET IDE (integrated development environment) provides an easy and rich debugging support. Once an exception occurs at run time, the program stops and the IDE marks the line which contains the error along with the details of that error and possible solutions. The runtime also provides built-in stack walking facilities making it much easier to locate bugs and error.

- **8. Simplified development:-**

- With .NET installing or uninstalling, a window-based application is a matter of copying or deleting files. This possible because .NET components are not referenced in the registry.

- **9. Framework class library:-**

- The framework class library (FCL) of the .NET framework contains a rich collection of classes that are available for developers to use these classes in code Microsoft has developed these classes to fulfill various tasks of applications, such as working with files and other data storages, performing input-output operations, web services, data access, and drawing graphics.
- The classes in the FCL are logically grouped under various namespaces such as system, System.collections, system.diagnostics, system.Globalization, system.IO, system.text etc.

## ● 10. Portability:-

- The application developed in the .NET environment is portable. When the source code of a program written in a CLR compliant language complies, it generates a machine-independent and intermediate code. This was originally called the Microsoft Intermediate Language (MSIL) and has now been renamed as the common Intermediate Language (CIL). CIL is the key to portability in .NET.

## Types & Object in .Net

The variables in C#, are categorized into the following types –

- Value types
- Reference types
- Pointer types

### Value Type

Value type variables can be assigned a value directly. They are derived from the class **System.ValueType**.

The value types directly contain data. Some examples are **int**, **char**, and **float**, which stores numbers, alphabets, and floating point numbers, respectively. When you declare an **int** type, the system allocates memory to store the value.

The following table lists the available value types in C# 2010 –

| Type | Represents             | Range         | Default Value |
|------|------------------------|---------------|---------------|
| bool | Boolean value          | True or False | False         |
| byte | 8-bit unsigned integer | 0 to 255      | 0             |

|         |                                                              |                                                                              |      |
|---------|--------------------------------------------------------------|------------------------------------------------------------------------------|------|
| char    | 16-bit Unicode character                                     | U +0000 to U +ffff                                                           | '\0' |
| decimal | 128-bit precise decimal values with 28-29 significant digits | $(-7.9 \times 10^{28} \text{ to } 7.9 \times 10^{28}) / 10^0 \text{ to } 28$ | 0.0M |
| double  | 64-bit double-precision floating point type                  | $(+/-)5.0 \times 10^{-324} \text{ to } (+/-)1.7 \times 10^{308}$             | 0.0D |
| float   | 32-bit single-precision floating point type                  | $-3.4 \times 10^{38} \text{ to } + 3.4 \times 10^{38}$                       | 0.0F |
| int     | 32-bit signed integer type                                   | -2,147,483,648 to 2,147,483,647                                              | 0    |
| long    | 64-bit signed integer type                                   | -9,223,372,036,854,775,808 to 9,223,372,036,854,775,807                      | 0L   |
| sbyte   | 8-bit signed integer type                                    | -128 to 127                                                                  | 0    |
| short   | 16-bit signed integer type                                   | -32,768 to 32,767                                                            | 0    |
| uint    | 32-bit unsigned integer type                                 | 0 to 4,294,967,295                                                           | 0    |

|        |                              |                                 |   |
|--------|------------------------------|---------------------------------|---|
| ulong  | 64-bit unsigned integer type | 0 to 18,446,744,073,709,551,615 | 0 |
| ushort | 16-bit unsigned integer type | 0 to 65,535                     | 0 |

To get the exact size of a type or a variable on a particular platform, you can use the **sizeof** method. The expression *sizeof(type)* yields the storage size of the object or type in bytes. Following is an example to get the size of *int* type on any machine –

```
using System;

namespace DataTypeApplication {
    class Program {
        static void Main(string[] args) {
            Console.WriteLine("Size of int: {0}", sizeof(int));
            Console.ReadLine();
        }
    }
}
```

When the above code is compiled and executed, it produces the following result –

### Reference Type

The reference types do not contain the actual data stored in a variable, but they contain a reference to the variables.

In other words, they refer to a memory location. Using multiple variables, the reference types can refer to a memory location. If the data in the memory location is changed by one of the variables, the other variable automatically reflects this change in value. Example of **built-in** reference types are: **object**, **dynamic**, and **string**.

## **Object Type**

The **Object Type** is the ultimate base class for all data types in C# Common Type System (CTS). Object is an alias for System.Object class. The object types can be assigned values of any other types, value types, reference types, predefined or user-defined types. However, before assigning values, it needs type conversion.

When a value type is converted to object type, it is called **boxing** and on the other hand, when an object type is converted to a value type, it is called **unboxing**.

```
object obj;  
obj = 100; // this is boxing
```

## **Dynamic Type**

You can store any type of value in the dynamic data type variable. Type checking for these types of variables takes place at run-time.

Syntax for declaring a dynamic type is –

```
dynamic <variable_name> = value;  
For example,
```

```
dynamic d = 20;
```

Dynamic types are similar to object types except that type checking for object type variables takes place at compile time, whereas that for the dynamic type variables takes place at run time.

## **String Type**

The **String Type** allows you to assign any string values to a variable. The string type is an alias for the System.String class. It is derived from object type. The value for a string type can be assigned using string literals in two forms: quoted and @quoted.

For example,

```
String str = "Tutorials Point";
```

A @quoted string literal looks as follows –

```
@"Tutorials Point";
```

The user-defined reference types are: class, interface, or delegate. We will discuss these types in later chapter.

## **Pointer Type**

Pointer type variables store the memory address of another type. Pointers in C# have the same capabilities as the pointers in C or C++.

Syntax for declaring a pointer type is –

```
type* identifier;
```

For example,

```
char* cptr;  
int* iptr;
```

We will discuss pointer types in the chapter 'Unsafe Codes'.

# **ASP.NET - Deployment**

---

There are two categories of ASP.NET deployment:

- **Local deployment** : In this case, the entire application is contained within a virtual directory and all the contents and assemblies are contained within it and available to the application.
- **Global deployment** : In this case, assemblies are available to every application running on the server.

There are different techniques used for deployment, however, we will discuss the following most common and easiest ways of deployment:

- **XCOPY deployment**
- **Copying a Website**
- **Creating a set up project**

## **XCOPY Deployment**

XCOPY deployment means making recursive copies of all the files to the target folder on the target machine. You can use any of the commonly used techniques:

- FTP transfer
- Using Server management tools that provide replication on a remote site
- MSI installer application

XCOPY deployment simply copies the application file to the production server and sets a virtual directory there. You need to set a virtual directory using the Internet Information Manager Microsoft Management Console (MMC snap-in).

The Copy Web Site option is available in Visual Studio. It is available from the Website -> Copy Web Site menu option. This menu item allows copying the current web site to another local or remote location. It is a sort of integrated FTP tool.

Using this option, you connect to the target destination, select the desired copy mode:

- Overwrite
- Source to Target Files
- Sync UP Source And Target Projects

Then proceed with copying the files physically. Unlike the XCOPY deployment, this process of deployment is done from Visual Studio environment. However, there are following problems with both the above deployment methods:

- You pass on your source code.
- There is no pre-compilation and related error checking for the files.
- The initial page load will be slow.

### **Creating a Setup Project**

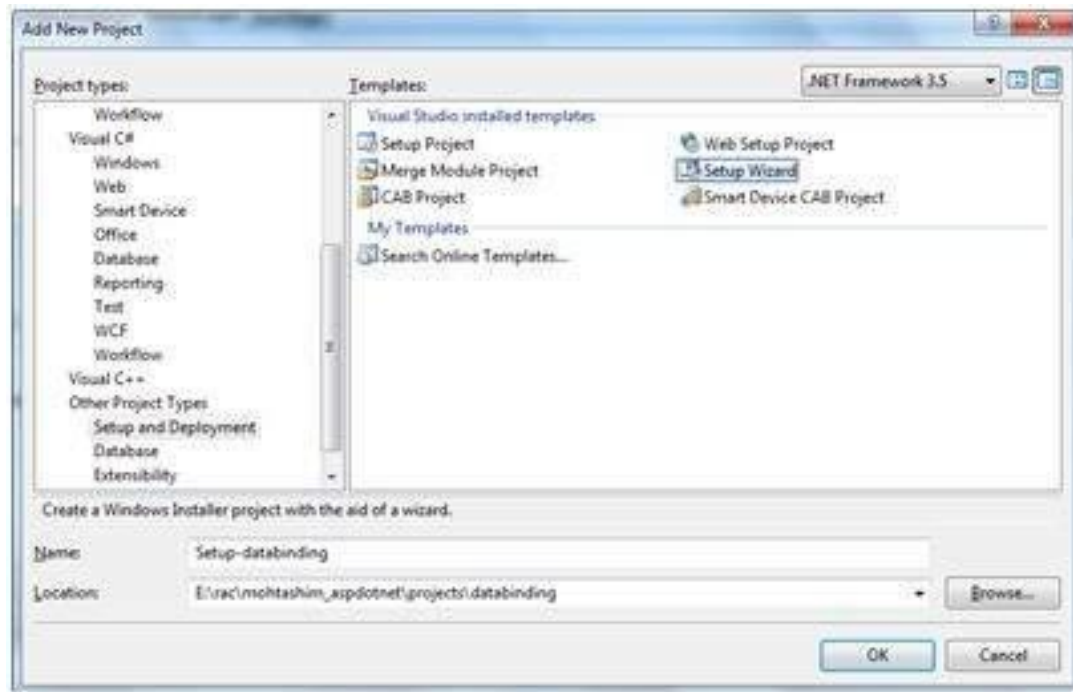
In this method, you use Windows Installer and package your web applications so it is ready to deploy on the production server. Visual Studio allows you to build deployment packages. Let us test this on one of our existing project, say the data binding project.

Open the project and take the following steps:



**Step (1)** : Select File -> Add -> New Project with the website root directory highlighted in the Solution Explorer.

**Step (2)** : Select Setup and Deployment, under Other Project Types. Select Setup Wizard.



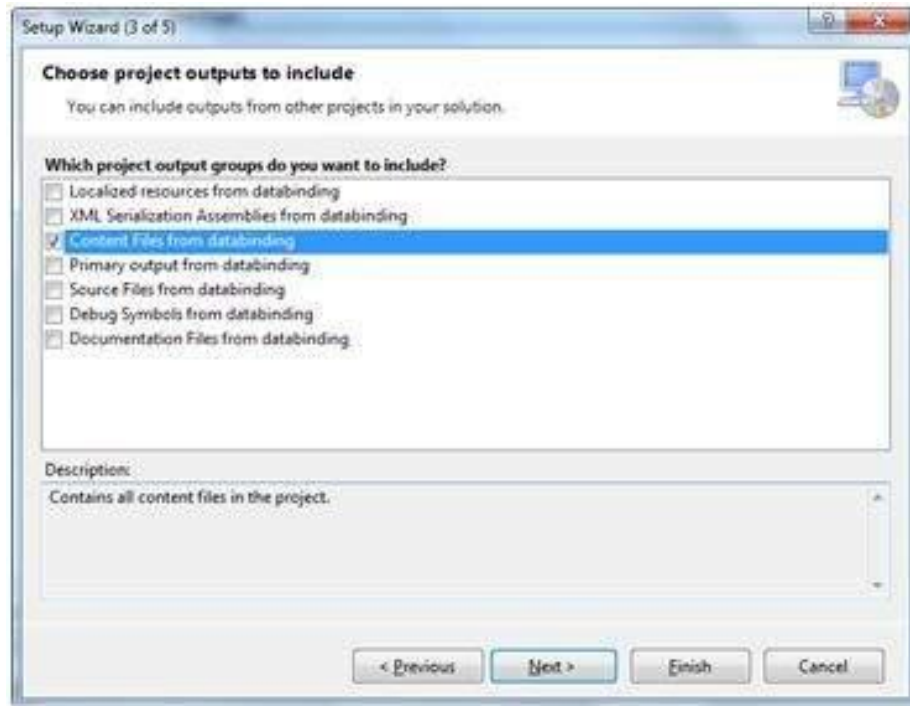
**Step (3)** : Choosing the default location ensures that the set up project will be located in its own folder under the root directory of the site. Click on okay to get the first splash screen of the wizard.



**Step (4) :** Choose a project type. Select 'Create a setup for a web application'.



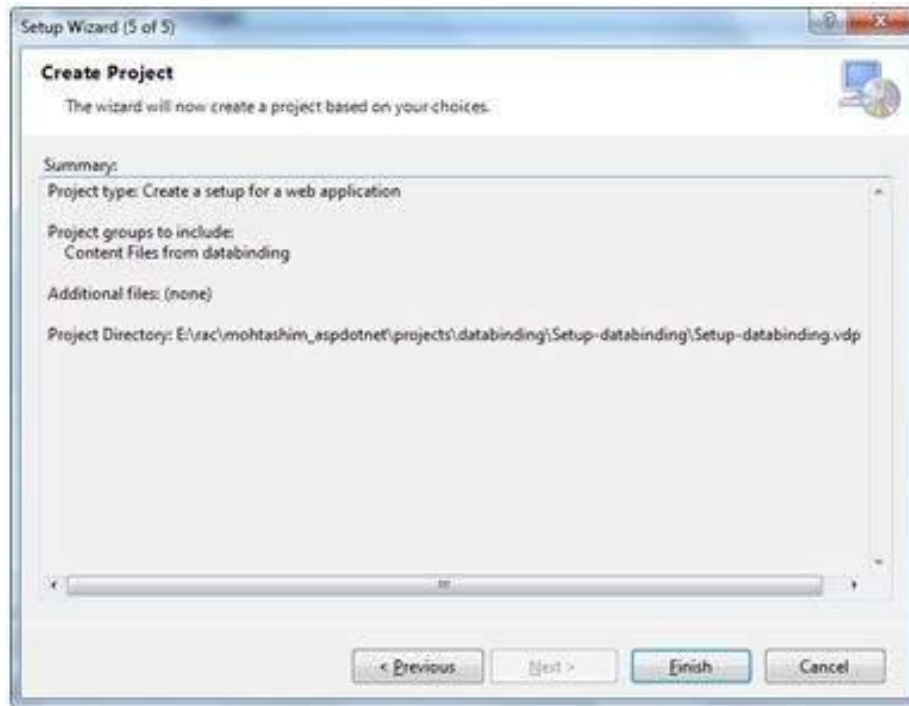
**Step (5) :** Next, the third screen asks to choose project outputs from all the projects in the solution. Check the check box next to 'Content Files from...'



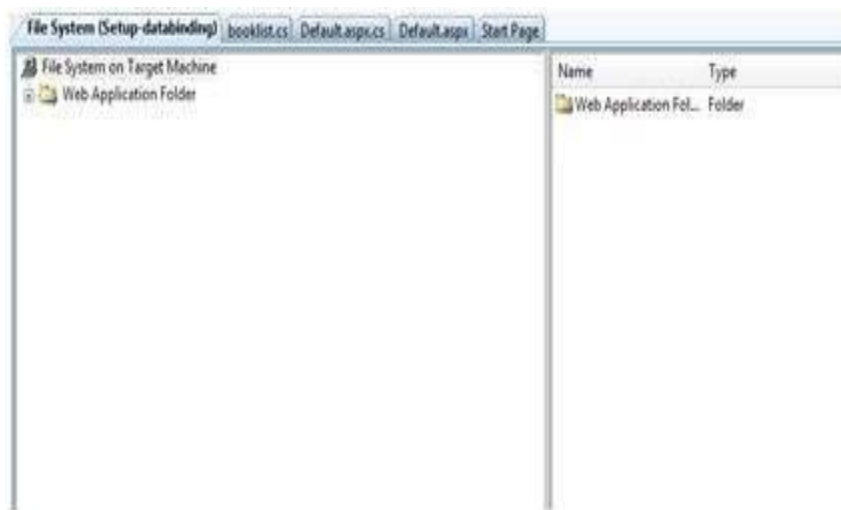
**Step (6) :** The fourth screen allows including other files like ReadMe. However, in our case there is no such file. Click on finish.



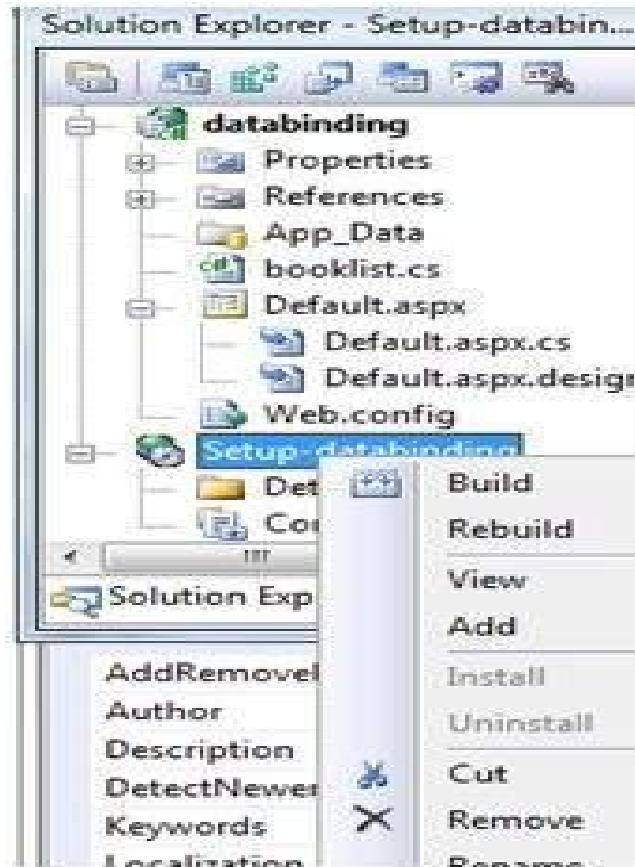
**Step (7) :** The final screen displays a summary of settings for the set up project.



**Step (8) :** The Set up project is added to the Solution Explorer and the main design window shows a file system editor.



**Step (9) :** Next step is to build the setup project. Right click on the project name in the Solution Explorer and select Build.



**Step (10) :** When build is completed, you get the following message in the Output window:

```
Packaging file 'Web.config'...
Packaging file 'Default.aspx'...
***** Build: 2 succeeded or up-to-date, 0 failed, 0 skipped *****
```

Two files are created by the build process:

- Setup.exe
- Setup-databinding.msi

You need to copy these files to the server. Double-click the setup file to install the content of the .msi file on the local machine.

## Objects in .NET

Since C# is an object-oriented language, program is designed using objects and classes in C#.

## C# Object

In C#, Object is a real world entity, for example, chair, car, pen, mobile, laptop etc.

In other words, object is an entity that has state and behavior. Here, state means data and behavior means functionality.

Object is a runtime entity, it is created at runtime.

Object is an instance of a class. All the members of the class can be accessed through object.

Let's see an example to create object using new keyword.

1. Student s1 = **new** Student();//creating an object of Student

In this example, Student is the type and s1 is the reference variable that refers to the instance of Student class. The new keyword allocates memory at runtime.

## Evolution of Web development



1. The 1990s
2. The 2000s
3. **The 2010s**
4. Conclusion

The Internet is one of the greatest inventions of all time. It has made our lives far easier and we are completely dependent on this asset to do even the smallest of our day-to-day activities. From shopping to making travel arrangements, entertaining ourselves, and more, the internet has a solution to everything.

What makes all of this possible? The web applications of course! Web applications allow us to interact with the internet and use all the services that we need.

Today, web applications not only have a beautiful user interface but also work efficiently with the latest technologies and get things done in seconds. However, initially web applications were very basic and had only a few design elements to work with.

Web development has come a long way since its inception and is now a huge field of study. Let's see how web development has evolved over time!

## **The 1990s**

HTML came into existence in the early 1990s as web developers realized the need for visually pleasing and understandable web pages that are easy to interact with. HTML still sits at the core of every web development.

The introduction of HTML allowed interconnection between web pages, well-structured page layouts, integration of images and animations in web pages.

Microsoft introduced AJAX technology in the year 1999 which used JavaScript to send data asynchronously.

## **The 2000s**

CSS (Cascading Style Sheets) came into being in the early 2000s. This allowed developers to create web pages with better visual elements like animations, graphics and more. Now, web pages could be personalized and made different from each other.

jQuery came into existence in 2006 and was invented by John Resig to upgrade the responsiveness of web pages.

After Apple launched the first generation of iPhones in 2007, web development expanded to the mobile platform as well. Now web developers had to develop websites that were responsive on mobile phones as well. Mobile web development got a new market as Apple released App Store in 2008.

## The 2010s

The Backbone JS framework, which was the first JavaScript framework, was introduced in 2010. This framework became very popular among developers and thus JavaScript programming gained more popularity.

The 2010s was the era of smartphones. Web development became more focused on enhancing security features and user-friendliness and reducing load time.

In 2014, **HTML5 and CSS3** were introduced which enabled developers to create better web pages with enhanced performance.

**Today, there exist multiple web development frameworks like AngularJS, Ruby on Rails, Django, etc. and programming languages like Python, Ruby, Java, etc. along with data formats like JSON and XML. Artificial Intelligence, Machine Learning, IoT and more such technologies are used in web development.**

## Conclusion

All these web development technologies allow us to develop creative, beautiful and highly efficient websites which look and work very differently from the web pages developed back when the internet was introduced.

Web development technologies keep advancing to meet the ever-changing needs of the users and to work well with all the technological signs of progress. Thus, we can expect web development to keep evolving and innovating in the future as well.